

Phase 4: Accounts & Subscriptions

User authentication and Stripe billing — where real company value gets tracked

Test with Stripe's test mode and test card numbers (4242 4242 4242 4242) before going live — never test billing code with real cards.

What this phase builds

User accounts (sign up / log in) and real subscription billing — the mechanism that actually tracks who's paying and updates your database, which is what a company's recurring revenue is built on.

1. Authentication — Supabase Auth

Don't hand-roll password storage or session security — use a managed provider. Supabase has a generous free tier and integrates cleanly with the Postgres database from Phase 2 (in fact, Supabase can host that database too).

■ login.html (snippet)

```
<!-- login.html – uses Supabase Auth (add to <head>) -->
<script src="https://cdn.jsdelivr.net/npm/@supabase/supabase-js@2"></script>

<script>
  const supabase = window.supabase.createClient(
    "https://YOUR_PROJECT.supabase.co",
    "YOUR_PUBLIC_ANON_KEY"
  );

  async function signUp(email, password) {
    const { data, error } = await supabase.auth.signUp({ email, password });
    if (error) { console.error(error); return; }
    console.log("Signed up:", data.user);
  }

  async function signIn(email, password) {
    const { data, error } = await supabase.auth.signInWithPassword({ email, password });
    if (error) { console.error(error); return; }
    // Store the session token to send with future API requests
    localStorage.setItem("access_token", data.session.access_token);
  }
</script>
```

2. Stripe — creating a subscription

In the Stripe Dashboard: create a Product, then a recurring Price (e.g. \$29/month). Copy that Price ID into your .env as STRIPE_PRICE_ID.

■ terminal

```
pip install stripe
```

■ billing.py

```
# billing.py – creating a Stripe subscription checkout session
import os
import stripe

stripe.api_key = os.getenv("STRIPE_SECRET_KEY")

PRICE_ID = os.getenv("STRIPE_PRICE_ID") # created in Stripe Dashboard: Product -> Price

def create_checkout_session(customer_email: str, success_url: str, cancel_url: str):
    session = stripe.checkout.Session.create(
        mode="subscription",
        payment_method_types=["card"],
        customer_email=customer_email,
        line_items=[{"price": PRICE_ID, "quantity": 1}],
        success_url=success_url,
        cancel_url=cancel_url,
    )
    return session.url # redirect the user's browser here
```

3. Checkout endpoint

Your front end calls this, then redirects the browser to the returned checkout_url — Stripe hosts the actual payment page.

■ main.py (addition)

```
# Add to main.py
from fastapi import Request
from billing import create_checkout_session

@app.post("/api/billing/checkout")
def start_checkout(email: str):
    url = create_checkout_session(
        customer_email=email,
        success_url="https://yoursite.com/success",
        cancel_url="https://yoursite.com/cancel",
    )
    return {"checkout_url": url}
```

4. Webhooks — the part that actually updates your database

This is the piece you were describing: when someone subscribes, Stripe calls this endpoint, and you flip `is_subscribed` to `True` on their `User` row. Set the webhook URL in the Stripe Dashboard once deployed (Phase 5).

■ `main.py` (addition)

```
# Add to main.py — Stripe tells you when payment events happen
import stripe
from sqlalchemy.orm import Session

STRIPE_WEBHOOK_SECRET = os.getenv("STRIPE_WEBHOOK_SECRET")

@app.post("/api/billing/webhook")
async def stripe_webhook(request: Request, db: Session = Depends(get_db)):
    payload = await request.body()
    sig_header = request.headers.get("stripe-signature")

    try:
        event = stripe.Webhook.construct_event(payload, sig_header, STRIPE_WEBHOOK_SECRET)
    except (ValueError, stripe.error.SignatureVerificationError):
        return Response(status_code=400)

    if event["type"] == "customer.subscription.created":
        customer_email = event["data"]["object"]["customer_email"]
        user = db.query(models.User).filter_by(email=customer_email).first()
        if user:
            user.is_subscribed = True
            db.commit()

    elif event["type"] == "customer.subscription.deleted":
        # subscription cancelled — flip the flag back off
        customer_email = event["data"]["object"].get("customer_email")
        user = db.query(models.User).filter_by(email=customer_email).first()
        if user:
            user.is_subscribed = False
            db.commit()

    return {"status": "received"}
```

5. Gating a feature behind subscription status

■ main.py (modify existing endpoint)

```
# Protecting a feature behind subscription status
from fastapi import HTTPException

@app.post("/api/audit/report") # now requires subscription
def download_report(payload: AuditIn, db: Session = Depends(get_db)):
    user = db.query(models.User).filter_by(email=payload.email).first()
    if not user or not user.is_subscribed:
        raise HTTPException(status_code=402, detail="Subscription required")

    result = run_audit(payload.tools)
    pdf_bytes = generate_report_pdf(payload.company, result)
    return Response(content=pdf_bytes, media_type="application/pdf")
```

What 'company value' actually is here

There's no separate 'value counter' to build — a company's value is derived from real metrics: how many rows in your User table have `is_subscribed = True`, multiplied by your price, gives you MRR (monthly recurring revenue). That number, tracked over time, is what investors and acquirers actually look at.